
Farmacia Archimede

Relazione del progetto di Tecnologie Web

Gruppo di Lavoro

Alessandro Bertolazzi 1227274
Guglielmo Barison 2042324

Informazioni sul sito

Indirizzo web: <https://tecweb.studenti.math.unipd.it/gbarison>
Email referente: guglielmo.barison@studenti.unipd.it

Accessi interni al sito

Ruolo	Email	Password
Utente	user@user.it	user
Amministratore	admin@admin.it	admin

Indice

1	Analisi	2
1.1	Descrizione del Sito	2
1.2	Analisi dell'utenza	2
2	Progettazione	2
2.1	Struttura del sito	2
2.2	Layout e contenuti delle pagine	3
2.3	Elementi comuni	3
2.3.1	Header	3
2.3.2	Footer	3
2.3.3	Breadcrumb	3
2.4	Emotional Design	3
2.5	Suddivisione del lavoro	3
3	Implementazione	3
3.1	PHP	3
3.2	Javascript	4
4	Sviluppi Futuri	4
5	Prestazioni e SEO	5
6	Accessibilità	5
6.1	Separazione tra contenuto, presentazione e struttura	5
6.1.1	Assenza di CSS e JavaScript	5
6.1.2	Contrasti	5
6.1.3	Lingue straniere e termini abbreviati	5
6.2	Strumenti di Testing	5
6.2.1	Ambienti di TESTING	5
6.2.2	Avvisi di validazione	5

1 Analisi

1.1 Descrizione del Sito

Il progetto nasce dall'esigenza di offrire un servizio farmaceutico moderno, accessibile e vicino alle esigenze di tutta la comunità. La farmacia si presenta come un punto di riferimento che coniuga la tradizione con l'innovazione, grazie alla realizzazione di un portale online all'avanguardia.

L'obiettivo principale è mettere la tecnologia al servizio della salute, garantendo un'esperienza digitale intuitiva per i più giovani e strumenti semplici e chiari per gli utenti meno esperti. Il sito è stato progettato per essere facilmente fruibile da persone di tutte le età, con particolare attenzione all'accessibilità: sono stati adottati standard che permettono la navigazione anche a chi presenta disabilità visive, motorie o cognitive.

Attraverso questa piattaforma, la farmacia vuole abbattere le barriere fisiche e digitali, offrendo servizi personalizzati, informazioni chiare e la possibilità di interagire direttamente con i professionisti del settore. In questo modo, si rafforza il legame con la comunità, promuovendo la salute come diritto fondamentale per tutti.

1.2 Analisi dell'utenza

L'utenza di una farmacia è estremamente eterogenea e comprende persone di tutte le età, condizioni sociali e livelli di alfabetizzazione digitale. Un'attenzione particolare va rivolta alle persone affette da disabilità, che rappresentano una parte significativa della popolazione e che spesso incontrano ostacoli nell'accesso ai servizi sanitari, sia fisici che digitali.

Le principali categorie di disabilità da considerare sono:

- **Disabilità visive:** includono ipovisione, cecità totale o parziale, daltonismo. Questi utenti necessitano di interfacce compatibili con screen reader, testi ad alto contrasto e possibilità di ingrandire i contenuti.
- **Disabilità motorie:** possono limitare l'uso di mouse o tastiera. È fondamentale garantire la navigazione tramite tastiera e prevedere pulsanti e link facilmente selezionabili.
- **Disabilità cognitive:** includono difficoltà di apprendimento, memoria o attenzione. L'interfaccia deve essere semplice, con istruzioni chiare e percorsi guidati.

Per rispondere alle esigenze di queste persone, il sito della farmacia è stato progettato seguendo le linee guida internazionali sull'accessibilità (WCAG), adottando soluzioni tecniche e grafiche che favoriscono l'inclusione. In questo modo, si garantisce a tutti gli utenti la possibilità di accedere ai servizi, alle informazioni sui farmaci e di interagire con i professionisti, riducendo il rischio di esclusione e promuovendo una reale equità nell'accesso alla salute.

2 Progettazione

2.1 Struttura del sito

Il sito non presenta un'elevata profondità di navigazione, in quanto quasi tutte le pagine sono raggiungibili dalla home page tramite il menu di navigazione principale. Le uniche eccezioni

sono rappresentate dalle pagine di dettaglio dei prodotti, che sono accessibili solo dalla pagina di elenco prodotti e quella dell'area personale, accessibile dopo il login.

- TODO

2.2 Layout e contenuti delle pagine

TODO

2.3 Elementi comuni

TODO

2.3.1 Header

TODO

2.3.2 Footer

TODO

2.3.3 Breadcrumb

TODO

2.4 Emotional Design

TODO

2.5 Suddivisione del lavoro

TODO

3 Implementazione

3.1 PHP

L'applicazione è sviluppata in PHP adottando una separazione netta tra presentazione e comportamento. Gli *entry point* risiedono in `public/` (ad es. `index.php`, `prodotti.php`, `prodotto.php`, `login.php`, `signup.php`, `logout.php`, `area_personale.php`) e hanno il solo compito di caricare l'autoloader e di delegare al servizio pertinente la costruzione della pagina. La logica applicativa è organizzata in `src/`.

Il rendering è orchestrato dalla classe `App\Core\PageBuilder`, che inizializza la sessione, verifica la disponibilità dei servizi essenziali (autenticazione e database) e compone l'output finale integrando *head*, *header*, *footer* e contenuto specifico della pagina. Le parti comuni dell'interfaccia sono costruite dai *builder* in `App\View` (`HeadBuilder`, `HeaderBuilder`, `FooterBuilder`), richiamati dal `PageBuilder` o dai servizi quando necessario. Questa scelta elimina duplicazioni tra pagine e consente modifiche centralizzate del layout.

La presentazione è definita da template HTML collocati in `src/html/`, privi di logica PHP. La sostituzione dei segnaposto è affidata al micro-motore `App\Core\Template`, che gestisce placeholder del tipo `{{ id }}` e blocchi `{{ id }} ...{{/id}}`, inclusi casi annidati e strutture array/oggetto.

L'accesso ai dati è incapsulato in `App\Core\Database`, che fornisce una connessione `mysqli` gestita come singleton. Le interrogazioni al database utilizzano *prepared statements* per prevenire SQL injection; per disaccoppiare persistenza e presentazione impieghiamo oggetti di trasferimento dati (DTO), come `App\Core\Model\ProductDTO` e `UserDTO`. Quando è necessario costruire elenchi filtrabili e paginati, le classi di supporto in `App\Core\Filter` e `App\Core\Pagination` compongono in maniera sistematica condizioni, limiti e offset, mantenendo le regole di business fuori dalle query ad hoc.

L'autenticazione è gestita da `App\Service\AuthService`, che utilizza `password_hash()` e `password_verify()` per il trattamento delle credenziali e si occupa della persistenza di sessione. I flussi di login e registrazione, coordinati rispettivamente da `LoginService` e `SignupService`, validano gli input lato server, impostano messaggi di errore e forniscono alla vista un modello coerente con lo stato dell'operazione. Nei form sensibili, in particolare nell'area riservata governata da `AreaPersonaleService`, introduciamo token anti-CSRF generati e verificati sul server; i messaggi di validazione sono esposti tramite elementi con `role="alert"` e `aria-live="assertive"` per una corretta fruizione con tecnologie assistive.

3.2 Javascript

Adottiamo un approccio di *progressive enhancement*: la pagina e i form funzionano anche senza script (invio tradizionale), mentre con JavaScript abilitiamo l'aggiornamento in-place della lista prodotti in risposta ai filtri. Al cambio dei controlli serializziamo i parametri con `FormData/URLSearchParams` e inviamo una richiesta `fetch` a `/prodotti.php`, marcando `#items` con `aria-busy` e usando l'header `X-Requested-With: fetch` per consentire al server di restituire, quando possibile, solo il frammento HTML della lista; la risposta, sia frammento sia pagina intera, viene parsata e reinserita in `#items`.

Le *validazioni* sono progettate in modo *server-authoritative*: la verifica effettiva dei parametri avviene sempre lato server e l'esito non dipende da ciò che succede nel browser. Quando un input non supera i controlli, il server restituisce un frammento HTML accessibile con i messaggi di errore, esposto con `role="alert"` e `aria-live="assertive"`, così che l'utente sia informato in modo chiaro e compatibile con le tecnologie assistive. In parallelo, gli *script di validazione lato client* affiancano il flusso senza sostituirlo: si agganciano agli eventi `input/change/submit`, normalizzano i valori, marcano i campi con `aria-invalid` e collegano i messaggi con `aria-describedby`. In questo modo l'esperienza resta fluida (aggiornamenti in-place quando possibile).

4 Sviluppi Futuri

IL CARRELLOOOOOOOOOOOOOO

5 Prestazioni e SEO

6 Accessibilità

TODO

6.1 Separazione tra contenuto, presentazione e struttura

TODO

6.1.1 Assenza di CSS e JavaScript

TODO

Attributi ARIA

TODO

6.1.2 Contrasti

TODO

6.1.3 Lingue straniere e termini abbreviati

TODO

6.2 Strumenti di Testing

TODO

6.2.1 Ambienti di TESTING

TODO

6.2.2 Avvisi di validazione

La validazione con Total Validator ha segnalato 5 falsi positivi:

- **E626** — **All element names must be in lower case.** Falso positivo dovuto agli SVG inline: alcuni nomi di elementi SVG sono in camelCase (p.es. `linearGradient`, `feGaussianBlur`) e sono validi secondo le specifiche SVG. Il validatore applica invece la regola XHTML/HTML che impone i nomi in minuscolo, segnalando erroneamente l’`<svg>`.

- **E621 — The ATTRIBUTE NAME attribute for this tag is missing.** Nel nostro caso riguarda `xmlns` mancante su `<html lang="it">`. Per documenti HTML5 serviti come `text/html` con `<!DOCTYPE html>`, il namespace è implicito e `xmlns` non è richiesto (né va usato).
- **E627 — All attribute names must be in lower case.** Come sopra, ma per gli attributi SVG: nomi come `viewBox`, `preserveAspectRatio` o `markerUnits` sono corretti in SVG; la segnalazione nasce perché lo strumento non riconosce il namespace SVG e pretende gli attributi tutti in minuscolo.
- **E61 — The ATTRIBUTE NAME attribute has a boolean value, and this may be ignored or cause errors in older browsers/robots.** L'avviso riguarda gli attributi booleani (p.es. `required`), che in HTML5 sono validi sia in forma ridotta (`required`) sia esplicita (`required="required"`). La presenza di una delle due forme è conforme; la segnalazione è quindi un falso positivo legato alla doppia notazione prevista dallo standard.